

Convolutional Networks

2026 年 6 月 10 日

1 卷积网络

到目前为止，我们已经研究了深度全连接网络，并利用它们探索了不同的优化策略和网络架构。全连接网络是实验的良好测试平台，因为它们计算效率很高，但在实际应用中，所有最先进的结果都使用卷积网络来实现。

首先，你将实现几种用于卷积网络的层类型。然后，你将使用这些层在 CIFAR-10 数据集上训练一个卷积网络。

```
[1]: # 配置 matplotlib 中文字体
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

# 设置单元格
import numpy as np # 导入 numpy 库, 用于数值计算
import matplotlib.pyplot as plt # 导入 matplotlib 库, 用于绘图
from cs231n.classifiers.cnn import * # 从 cs231n.classifiers.cnn 导入所有内容
(卷积神经网络相关类/函数)

from cs231n.data_utils import get_CIFAR10_data # 从数据工具模块导入获取 CIFAR-10
数据的函数
from cs231n.gradient_check import eval_numerical_gradient_array, \
    eval_numerical_gradient # 导入梯度检查相关函数
from cs231n.layers import * # 导入基础层模块中的所有内容
from cs231n.fast_layers import * # 导入快速层模块中的所有内容 (可能是优化过的实现)

from cs231n.solver import Solver # 导入求解器类, 用于训练模型
```

```

# 设置 matplotlib 在 notebook 中内联显示
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # 设置图表默认大小
plt.rcParams['image.interpolation'] = 'nearest' # 设置图像插值方式为最近邻
plt.rcParams['image.cmap'] = 'gray' # 设置默认图像颜色映射为灰度

# 用于自动重新加载外部模块
# 参考 http://stackoverflow.com/questions/1907993/
    ↳ autoreload-of-modules-in-ipython
# 加载自动重载扩展
%load_ext autoreload
# 设置自动重载：当外部模块修改时，自动重新导入
%autoreload 2

def rel_error(x, y):
    """ 返回相对误差 """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y)))) # 计算 x 和 y 之间的相对误差（避免除以零）

```

```

[2]: # 加载（预处理后的）CIFAR-10 数据
data = get_CIFAR10_data()
# 遍历数据字典中的所有键值对并打印
for k, v in list(data.items()):
    print(f"{k}: {v.shape}") # 打印每个数据子集的名称及其形状（维度信息）

```

```

X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)

```

2 卷积：朴素前向传播

卷积网络的核心是卷积操作。在文件 `cs231n/layers.py` 中，在函数 `conv_forward_naive` 里实现卷积层的前向传播。

此时你不必过于担心效率问题，只需以你认为最清晰的方式编写代码即可。

你可以通过运行以下代码来测试你的实现：

```
[3]: # 定义输入和权重的形状
# x_shape: (批量大小, 输入通道数, 高度, 宽度)
x_shape = (2, 3, 4, 4)
# w_shape: (输出通道数, 输入通道数, 卷积核高度, 卷积核宽度)
w_shape = (3, 3, 4, 4)

# 生成输入数据 x: 从-0.1 到 0.5 均匀分布的数值, 重塑为 x_shape 的形状
x = np.linspace(-0.1, 0.5, num=np.prod(x_shape)).reshape(x_shape)
# 生成权重 w: 从-0.2 到 0.3 均匀分布的数值, 重塑为 w_shape 的形状
w = np.linspace(-0.2, 0.3, num=np.prod(w_shape)).reshape(w_shape)
# 生成偏置 b: 从-0.1 到 0.2 均匀分布的 3 个数值 (与输出通道数一致)
b = np.linspace(-0.1, 0.2, num=3)

# 卷积参数: 步长为 2, 填充为 1
conv_param = {'stride': 2, 'pad': 1}
# 执行朴素卷积前向传播, 得到输出 out 和缓存 (未使用)
out, _ = conv_forward_naive(x, w, b, conv_param)

# 正确的输出结果 (用于验证)
correct_out = np.array([[[[-0.08759809, -0.10987781],
                           [-0.18387192, -0.2109216 ]],
                          [[ 0.21027089,  0.21661097],
                           [ 0.22847626,  0.23004637]],
                          [[ 0.50813986,  0.54309974],
                           [ 0.64082444,  0.67101435]]],
                         [[[-0.98053589, -1.03143541],
                           [-1.19128892, -1.24695841]],
                          [[ 0.69108355,  0.66880383],
                           [ 0.59480972,  0.56776003]],
                          [[ 2.36270298,  2.36904306],
                           [ 2.38090835,  2.38247847]]]])

# 将你的输出与正确输出进行比较; 差异应在 1e-8 左右
print('测试 conv_forward_naive')
print('差异: ', rel_error(out, correct_out))
```

测试 `conv_forward_naive`

差异: `2.2121476417505994e-08`

2.1 补充说明：通过卷积进行图像处理

作为一种既可以检验你的实现，又能帮助你更好地理解卷积层所能执行的操作类型的有趣方式，我们将创建一个包含两张图像的输入，并手动设置一些滤波器来执行常见的图像处理操作（灰度转换和边缘检测）。卷积前向传播会将这些操作应用到每张输入图像上。然后我们可以将结果可视化，以此作为合理性检查。

```
[4]: from imageio import imread # 从 imageio 导入 imread 函数, 用于读取图像
from PIL import Image # 从 PIL 导入 Image 类, 用于图像处理

# 读取小猫和小狗的图像
kitten = imread('cs231n/notebook_images/kitten.jpg')
puppy = imread('cs231n/notebook_images/puppy.jpg')

# 小猫图像较宽, 裁剪成正方形 (去除左右两侧多余部分)
d = kitten.shape[1] - kitten.shape[0] # 计算宽度与高度的差值
kitten_cropped = kitten[:, d//2:-d//2, :] # 居中裁剪, 使宽高相等

img_size = 200 # 图像尺寸 (如果运行太慢可以调小)
# 调整小狗和小猫图像的大小为 img_size x img_size
resized_puppy = np.array(Image.fromarray(puppy).resize((img_size, img_size)))
resized_kitten = np.array(Image.fromarray(kitten_cropped).resize((img_size,
    img_size)))

# 创建输入数组 x, 形状为 (批量大小, 通道数, 高度, 宽度)
x = np.zeros((2, 3, img_size, img_size))
# 将小狗图像的通道维度移到前面 (PIL 图像是 HWC 格式, 转为 CHW 格式)
x[0, :, :, :] = resized_puppy.transpose((2, 0, 1))
# 将小猫图像的通道维度移到前面
x[1, :, :, :] = resized_kitten.transpose((2, 0, 1))

# 设置卷积权重 w, 包含 2 个滤波器, 每个滤波器大小为 3x3x3 (输入通道数为 3)
w = np.zeros((2, 3, 3, 3))
```

```

# 第一个滤波器：将图像转换为灰度图
# 设置红、绿、蓝三个通道的权重（符合灰度转换公式： $0.3*R + 0.6*G + 0.1*B$ ）
w[0, 0, :, :] = [[0, 0, 0], [0, 0.3, 0], [0, 0, 0]] # 红色通道权重
w[0, 1, :, :] = [[0, 0, 0], [0, 0.6, 0], [0, 0, 0]] # 绿色通道权重
w[0, 2, :, :] = [[0, 0, 0], [0, 0.1, 0], [0, 0, 0]] # 蓝色通道权重

# 第二个滤波器：检测蓝色通道中的水平边缘（使用 Sobel 算子）
w[1, 2, :, :] = [[1, 2, 1], [0, 0, 0], [-1, -2, -1]] # 蓝色通道的水平边缘检测核

# 偏置向量 b：灰度转换不需要偏置，边缘检测需要加 128 避免输出为负
b = np.array([0, 128])

# 执行卷积前向传播：对 x 中的每个输入应用 w 中的每个滤波器，加上偏置 b
out, _ = conv_forward_naive(x, w, b, {'stride': 1, 'pad': 1}) # 步长为 1，填充为 1

def imshow_no_ax(img, normalize=True):
    """ 简易图像显示函数：将图像转为 uint8 格式并去除坐标轴 """
    if normalize:
        # 归一化到 0-255 范围
        img_max, img_min = np.max(img), np.min(img)
        img = 255.0 * (img - img_min) / (img_max - img_min)
    plt.imshow(img.astype('uint8')) # 显示图像（转为 uint8 格式）
    plt.gca().axis('off') # 关闭坐标轴

# 显示原始图像和卷积操作的结果
plt.subplot(2, 3, 1) # 第 1 行第 1 列子图
imshow_no_ax(puppy, normalize=False) # 显示原始小狗图像（无需归一化）
plt.title('原始图像')
plt.subplot(2, 3, 2) # 第 1 行第 2 列子图
imshow_no_ax(out[0, 0]) # 显示小狗图像的灰度结果
plt.title('灰度图')
plt.subplot(2, 3, 3) # 第 1 行第 3 列子图
imshow_no_ax(out[0, 1]) # 显示小狗图像的边缘检测结果

```

```
plt.title('边缘检测')

plt.subplot(2, 3, 4) # 第 2 行第 1 列子图
imshow_no_ax(kitten_cropped, normalize=False) # 显示原始小猫图像
plt.subplot(2, 3, 5) # 第 2 行第 2 列子图
imshow_no_ax(out[1, 0]) # 显示小猫图像的灰度结果
plt.subplot(2, 3, 6) # 第 2 行第 3 列子图
imshow_no_ax(out[1, 1]) # 显示小猫图像的边缘检测结果

plt.show() # 显示所有子图
```

C:\Users\keymo\AppData\Local\Temp\ipykernel_31644\1025854422.py:5:

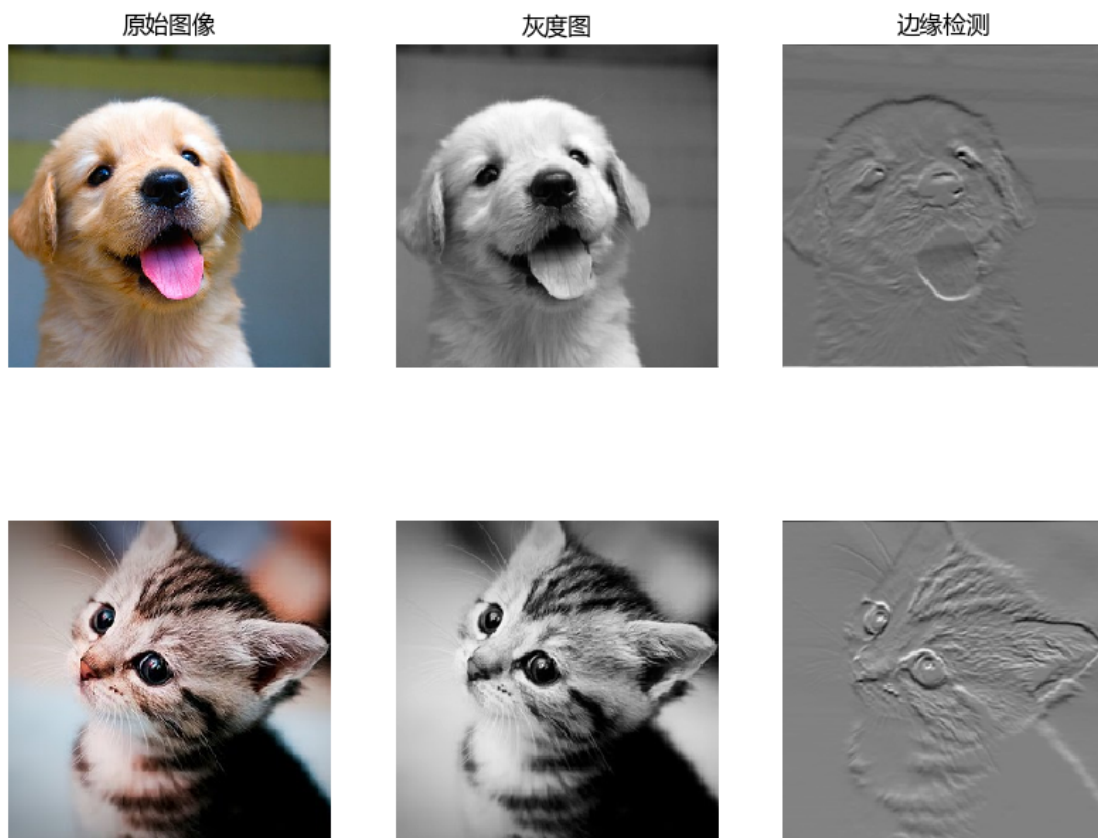
DeprecationWarning: Starting with ImageIO v3 the behavior of this function will switch to that of `iio.v3.imread`. To keep the current behavior (and make this warning disappear) use ``import imageio.v2 as imageio`` or call ``imageio.v2.imread`` directly.

```
kitten = imread('cs231n/notebook_images/kitten.jpg')
```

C:\Users\keymo\AppData\Local\Temp\ipykernel_31644\1025854422.py:6:

DeprecationWarning: Starting with ImageIO v3 the behavior of this function will switch to that of `iio.v3.imread`. To keep the current behavior (and make this warning disappear) use ``import imageio.v2 as imageio`` or call ``imageio.v2.imread`` directly.

```
puppy = imread('cs231n/notebook_images/puppy.jpg')
```



3 卷积：朴素反向传播

在文件 `cs231n/layers.py` 中的函数 `conv_backward_naive` 里实现卷积操作的反向传播。同样，你不必过于担心计算效率问题。

完成后，运行以下代码，通过数值梯度检查来验证你的反向传播实现是否正确。

```
[5]: np.random.seed(231) # 设置随机种子，保证结果可复现
# 生成随机输入数据 x: 形状为 (批量大小, 输入通道数, 高度, 宽度)
x = np.random.randn(4, 3, 5, 5)
# 生成随机权重 w: 形状为 (输出通道数, 输入通道数, 卷积核高度, 卷积核宽度)
w = np.random.randn(2, 3, 3, 3)
# 生成随机偏置 b: 长度为输出通道数
b = np.random.randn(2,)
# 生成随机的输出梯度 dout: 形状与卷积输出一致
dout = np.random.randn(4, 2, 5, 5)
```

```

# 卷积参数: 步长为 1, 填充为 1
conv_param = {'stride': 1, 'pad': 1}

# 使用数值方法计算  $x$ 、 $w$ 、 $b$  的梯度 (用于验证)
# 计算  $x$  的数值梯度
dx_num = eval_numerical_gradient_array(lambda x: conv_forward_naive(x, w, b,
    ↪conv_param)[0], x, dout)
# 计算  $w$  的数值梯度
dw_num = eval_numerical_gradient_array(lambda w: conv_forward_naive(x, w, b,
    ↪conv_param)[0], w, dout)
# 计算  $b$  的数值梯度
db_num = eval_numerical_gradient_array(lambda b: conv_forward_naive(x, w, b,
    ↪conv_param)[0], b, dout)

# 执行卷积前向传播, 获取输出和缓存 (用于反向传播)
out, cache = conv_forward_naive(x, w, b, conv_param)
# 执行卷积反向传播, 计算  $dx$ 、 $dw$ 、 $db$ 
dx, dw, db = conv_backward_naive(dout, cache)

# 你的误差应在  $1e-8$  或更小范围内
print('测试 conv_backward_naive 函数')
print('dx 误差: ', rel_error(dx, dx_num))
print('dw 误差: ', rel_error(dw, dw_num))
print('db 误差: ', rel_error(db, db_num))

```

测试 conv_backward_naive 函数

```

dx 误差:  1.159803161159293e-08
dw 误差:  2.2471264748452487e-10
db 误差:  3.37264006649648e-11

```

4 最大池化: 朴素前向传播

在文件 `cs231n/layers.py` 中的函数 `max_pool_forward_naive` 里实现最大池化操作的前向传播。同样, 不必过于担心计算效率问题。

运行以下代码来检验你的实现:


```
[6]: # 定义输入  $x$  的形状: (批量大小, 通道数, 高度, 宽度)
x_shape = (2, 3, 4, 4)
# 生成输入数据  $x$ : 从 -0.3 到 0.4 均匀分布的数值, 重塑为  $x\_shape$  的形状
x = np.linspace(-0.3, 0.4, num=np.prod(x_shape)).reshape(x_shape)
# 池化参数: 池化窗口宽度为 2, 高度为 2, 步长为 2
pool_param = {'pool_width': 2, 'pool_height': 2, 'stride': 2}

# 执行朴素最大池化前向传播, 得到输出  $out$  和缓存 (未使用)
out, _ = max_pool_forward_naive(x, pool_param)

# 正确的输出结果 (用于验证)
correct_out = np.array([[[[-0.26315789, -0.24842105],
                           [-0.20421053, -0.18947368]],
                          [[-0.14526316, -0.13052632],
                           [-0.08631579, -0.07157895]],
                          [[-0.02736842, -0.01263158],
                           [ 0.03157895,  0.04631579]]],
                        [[[ 0.09052632,  0.10526316],
                          [ 0.14947368,  0.16421053]],
                         [[ 0.20842105,  0.22315789],
                          [ 0.26736842,  0.28210526]],
                         [[ 0.32631579,  0.34105263],
                          [ 0.38526316,  0.4          ]]]]])

# 将你的输出与正确输出进行比较。差异应在  $1e-8$  量级。
print('测试 max_pool_forward_naive 函数: ')
print('差异: ', rel_error(out, correct_out))
```

测试 max_pool_forward_naive 函数:

差异: 4.1666665157267834e-08

5 最大池化: 朴素反向传播

在文件 `cs231n/layers.py` 中的函数 `max_pool_backward_naive` 里实现最大池化操作的反向传播。你无需担心计算效率问题。

运行以下代码, 通过数值梯度检查来验证你的实现:

```
[7]: np.random.seed(231) # 设置随机种子, 确保结果可复现
# 生成随机输入数据 x: 形状为 (批量大小, 通道数, 高度, 宽度)
x = np.random.randn(3, 2, 8, 8)
# 生成随机的输出梯度 dout: 形状与最大池化输出一致
dout = np.random.randn(3, 2, 4, 4)
# 池化参数: 池化窗口高度为 2, 宽度为 2, 步长为 2
pool_param = {'pool_height': 2, 'pool_width': 2, 'stride': 2}

# 使用数值方法计算 x 的梯度 (用于验证)
dx_num = eval_numerical_gradient_array(lambda x: max_pool_forward_naive(x, pool_param), x, dout)

# 执行最大池化前向传播, 获取输出和缓存 (用于反向传播)
out, cache = max_pool_forward_naive(x, pool_param)
# 执行最大池化反向传播, 计算 dx
dx = max_pool_backward_naive(dout, cache)

# 你的误差应在 1e-12 量级
print('测试 max_pool_backward_naive 函数: ')
print('dx 误差: ', rel_error(dx, dx_num))
```

测试 max_pool_backward_naive 函数:

dx 误差: 3.27562514223145e-12

6 快速层

让卷积层和池化层变得快速可能具有挑战性。为了省去你的麻烦, 我们在文件 `cs231n/fast_layers.py` 中提供了卷积层和池化层的前向传播与反向传播的快速实现。

6.0.1 执行下面的单元格, 保存笔记本, 然后重启运行时

快速卷积实现依赖于一个 Cython 扩展; 要编译它, 请运行下面的单元格。接下来, 保存 Colab 笔记本 (File > Save) 并**重启运行时** (Runtime > Restart runtime)。之后你可以从上到下重新执行前面的单元格, 并跳过下面的单元格, 因为你只需要为编译步骤运行一次它。

```
[ ]: # 在 notebook 中运行: 自动查找并编译 assignment2 下的 cs231n 扩展
import os, sys
```

```

def find_cs231n_dir(root=os.getcwd()):
    for dirpath, dirnames, filenames in os.walk(root):
        if 'cs231n' in dirnames:
            candidate = os.path.join(dirpath, 'cs231n')
            if os.path.exists(os.path.join(candidate, 'setup.py')):
                return candidate
    return None

cs231n_dir = find_cs231n_dir()
if cs231n_dir is None:
    print('未在当前工作目录及子目录中找到包含 setup.py 的 cs231n 文件夹。请确认，
    ↪notebook 的工作目录，然后手动运行编译命令。')
else:
    print('在目录编译：', cs231n_dir)
    orig_cwd = os.getcwd()
    os.chdir(cs231n_dir)
    # 使用当前 Python 解释器以避免环境不一致
    !{sys.executable} setup.py build_ext --inplace
    os.chdir(orig_cwd)
    print('编译完成；如果成功，请重启内核并从头运行笔记本（或继续执行剩余单元格）。')

```

卷积层和池化层的快速版本的编程接口与你上面实现的朴素版本完全相同：前向传播接收数据、权重和参数，生成输出和缓存对象；反向传播接收上游导数和缓存对象，生成关于数据和权重的梯度。

注意：池化的快速实现只有在池化区域不重叠且能完整覆盖输入时才能达到最佳性能。如果不满足这些条件，那么快速池化实现不会比朴素实现快多少。

你可以通过运行以下代码来比较这些层的朴素版本和快速版本的性能：

```

[14]: # 相对误差应在 1e-9 或更小范围内
from cs231n.fast_layers import conv_forward_fast, conv_backward_fast # 导入快速
快速卷积的前向和反向传播函数
from time import time # 导入时间模块，用于计时
np.random.seed(231) # 设置随机种子，保证结果可复现

# 生成随机输入数据 x: (批量大小 100, 输入通道 3, 高度 31, 宽度 31)
x = np.random.randn(100, 3, 31, 31)
# 生成随机权重 w: (输出通道 25, 输入通道 3, 卷积核 3x3)

```

```

w = np.random.randn(25, 3, 3, 3)
# 生成随机偏置 b: 长度为输出通道数 25
b = np.random.randn(25,)
# 生成随机的输出梯度 dout: 形状与卷积输出一致
dout = np.random.randn(100, 25, 16, 16)
# 卷积参数: 步长 2, 填充 1
conv_param = {'stride': 2, 'pad': 1}

# 测试卷积前向传播的速度和正确性
t0 = time() # 记录开始时间
# 朴素卷积前向传播
out_naive, cache_naive = conv_forward_naive(x, w, b, conv_param)
t1 = time() # 记录朴素方法结束时间
# 快速卷积前向传播
out_fast, cache_fast = conv_forward_fast(x, w, b, conv_param)
t2 = time() # 记录快速方法结束时间

print('测试 conv_forward_fast: ')
print('朴素方法: %fs' % (t1 - t0)) # 打印朴素方法耗时
print('快速方法: %fs' % (t2 - t1)) # 打印快速方法耗时
print('加速比: %fx' % ((t1 - t0) / (t2 - t1))) # 计算并打印加速比
print('输出差异: ', rel_error(out_naive, out_fast)) # 验证输出一致性

# 测试卷积反向传播的速度和正确性
t0 = time() # 记录开始时间
# 朴素卷积反向传播
dx_naive, dw_naive, db_naive = conv_backward_naive(dout, cache_naive)
t1 = time() # 记录朴素方法结束时间
# 快速卷积反向传播
dx_fast, dw_fast, db_fast = conv_backward_fast(dout, cache_fast)
t2 = time() # 记录快速方法结束时间

print('\n测试 conv_backward_fast: ')
print('朴素方法: %fs' % (t1 - t0)) # 打印朴素方法耗时
print('快速方法: %fs' % (t2 - t1)) # 打印快速方法耗时
print('加速比: %fx' % ((t1 - t0) / (t2 - t1))) # 计算并打印加速比

```

```
print('dx 差异: ', rel_error(dx_naive, dx_fast)) # 验证 dx 一致性
print('dw 差异: ', rel_error(dw_naive, dw_fast)) # 验证 dw 一致性
print('db 差异: ', rel_error(db_naive, db_fast)) # 验证 db 一致性
```

测试 conv_forward_fast:

朴素方法: 2.291019s

快速方法: 0.007657s

加速比: 299.222457x

输出差异: 2.95584471112945e-11

测试 conv_backward_fast:

朴素方法: 3.003622s

快速方法: 0.008996s

加速比: 333.892608x

dx 差异: 1.0692258445766582e-11

dw 差异: 5.617985857421359e-13

db 差异: 0.0

```
[15]: # 相对误差应接近 0.0
from cs231n.fast_layers import max_pool_forward_fast, max_pool_backward_fast #
    ↪ 导入快速最大池化的前向和反向传播函数
np.random.seed(231) # 设置随机种子, 确保结果可复现

# 生成随机输入数据 x: (批量大小 100, 通道数 3, 高度 32, 宽度 32)
x = np.random.randn(100, 3, 32, 32)
# 生成随机的输出梯度 dout: 形状与最大池化输出一致
dout = np.random.randn(100, 3, 16, 16)
# 池化参数: 池化窗口高度 2, 宽度 2, 步长 2
pool_param = {'pool_height': 2, 'pool_width': 2, 'stride': 2}

# 测试最大池化前向传播的速度和正确性
t0 = time() # 记录开始时间
# 朴素最大池化前向传播
out_naive, cache_naive = max_pool_forward_naive(x, pool_param)
t1 = time() # 记录朴素方法结束时间
# 快速最大池化前向传播
out_fast, cache_fast = max_pool_forward_fast(x, pool_param)
```

```

t2 = time() # 记录快速方法结束时间

print('测试 pool_forward_fast: ')
print('朴素方法: %fs' % (t1 - t0)) # 打印朴素方法耗时
print('快速方法: %fs' % (t2 - t1)) # 打印快速方法耗时
print('加速比: %fx' % ((t1 - t0) / (t2 - t1))) # 计算并打印加速比
print('输出差异: ', rel_error(out_naive, out_fast)) # 验证输出一致性

# 测试最大池化反向传播的速度和正确性
t0 = time() # 记录开始时间
# 朴素最大池化反向传播
dx_naive = max_pool_backward_naive(dout, cache_naive)
t1 = time() # 记录朴素方法结束时间
# 快速最大池化反向传播
dx_fast = max_pool_backward_fast(dout, cache_fast)
t2 = time() # 记录快速方法结束时间

print('\n测试 pool_backward_fast: ')
print('朴素方法: %fs' % (t1 - t0)) # 打印朴素方法耗时
print('快速方法: %fs' % (t2 - t1)) # 打印快速方法耗时
print('加速比: %fx' % ((t1 - t0) / (t2 - t1))) # 计算并打印加速比
print('dx 差异: ', rel_error(dx_naive, dx_fast)) # 验证 dx 一致性

```

测试 pool_forward_fast:

朴素方法: 0.181592s

快速方法: 0.003057s

加速比: 59.392701x

输出差异: 0.0

测试 pool_backward_fast:

朴素方法: 0.520117s

快速方法: 0.008939s

加速比: 58.184941x

dx 差异: 0.0

7 卷积“三明治”层

在上一次作业中，我们介绍了“三明治”层的概念，它将多个操作组合成常用的模式。在文件 `cs231n/layer_utils.py` 中，你会找到一些三明治层，它们实现了卷积网络中几种常用的模式。运行下面的单元格来验证它们的使用是否正确。

```
[16]: from cs231n.layer_utils import conv_relu_pool_forward, conv_relu_pool_backward
      ↪ # 导入卷积-激活-池化组合层的前向和反向传播函数

      np.random.seed(231) # 设置随机种子，保证结果可复现

      # 生成随机输入数据 x: (批量大小 2, 输入通道 3, 高度 16, 宽度 16)
      x = np.random.randn(2, 3, 16, 16)
      # 生成随机权重 w: (输出通道 3, 输入通道 3, 卷积核 3x3)
      w = np.random.randn(3, 3, 3, 3)
      # 生成随机偏置 b: 长度为输出通道数 3
      b = np.random.randn(3,)
      # 生成随机的输出梯度 dout: 形状与组合层输出一致
      dout = np.random.randn(2, 3, 8, 8)
      # 卷积参数: 步长 1, 填充 1
      conv_param = {'stride': 1, 'pad': 1}
      # 池化参数: 池化窗口 2x2, 步长 2
      pool_param = {'pool_height': 2, 'pool_width': 2, 'stride': 2}

      # 执行卷积-激活-池化组合层的前向传播
      out, cache = conv_relu_pool_forward(x, w, b, conv_param, pool_param)
      # 执行组合层的反向传播，计算梯度
      dx, dw, db = conv_relu_pool_backward(dout, cache)

      # 使用数值方法计算 x、w、b 的梯度 (用于验证)
      dx_num = eval_numerical_gradient_array(lambda x: conv_relu_pool_forward(x, w,
      ↪ b, conv_param, pool_param)[0], x, dout)
      dw_num = eval_numerical_gradient_array(lambda w: conv_relu_pool_forward(x, w,
      ↪ b, conv_param, pool_param)[0], w, dout)
      db_num = eval_numerical_gradient_array(lambda b: conv_relu_pool_forward(x, w,
      ↪ b, conv_param, pool_param)[0], b, dout)

      # 相对误差应在 1e-8 或更小范围内
```

```

print('测试 conv_relu_pool 组合层')
print('dx 误差: ', rel_error(dx_num, dx))
print('dw 误差: ', rel_error(dw_num, dw))
print('db 误差: ', rel_error(db_num, db))

```

测试 conv_relu_pool 组合层

dx 误差: 4.397502834267091e-09

dw 误差: 3.651699397290073e-09

db 误差: 7.054812624223923e-10

```

[17]: from cs231n.layer_utils import conv_relu_forward, conv_relu_backward # 导入卷
      积-激活 (ReLU) 组合层的前向和反向传播函数
      np.random.seed(231) # 设置随机种子, 确保结果可复现

      # 生成随机输入数据 x: (批量大小 2, 输入通道 3, 高度 8, 宽度 8)
      x = np.random.randn(2, 3, 8, 8)
      # 生成随机权重 w: (输出通道 3, 输入通道 3, 卷积核 3x3)
      w = np.random.randn(3, 3, 3, 3)
      # 生成随机偏置 b: 长度为输出通道数 3
      b = np.random.randn(3,)
      # 生成随机的输出梯度 dout: 形状与卷积-激活层输出一致
      dout = np.random.randn(2, 3, 8, 8)
      # 卷积参数: 步长 1, 填充 1
      conv_param = {'stride': 1, 'pad': 1}

      # 执行卷积-激活 (ReLU) 组合层的前向传播
      out, cache = conv_relu_forward(x, w, b, conv_param)
      # 执行组合层的反向传播, 计算梯度
      dx, dw, db = conv_relu_backward(dout, cache)

      # 使用数值方法计算 x、w、b 的梯度 (用于验证)
      dx_num = eval_numerical_gradient_array(lambda x: conv_relu_forward(x, w, b,
      ↪conv_param)[0], x, dout)
      dw_num = eval_numerical_gradient_array(lambda w: conv_relu_forward(x, w, b,
      ↪conv_param)[0], w, dout)
      db_num = eval_numerical_gradient_array(lambda b: conv_relu_forward(x, w, b,
      ↪conv_param)[0], b, dout)

```



```
# 相对误差应在 1e-8 或更小范围内
print('测试 conv_relu 组合层: ')
print('dx 误差: ', rel_error(dx_num, dx))
print('dw 误差: ', rel_error(dw_num, dw))
print('db 误差: ', rel_error(db_num, db))
```

测试 conv_relu 组合层:

dx 误差: 8.03522627181292e-09

dw 误差: 2.0902405745264502e-10

db 误差: 3.287958402642519e-10

8 三层卷积网络

既然你已经实现了所有必要的层，我们可以将它们组合成一个简单的卷积网络。

打开文件 `cs231n/classifiers/cnn.py`，完成 `ThreeLayerConvNet` 类的实现。记住，你可以在实现中使用快速层/三明治层（已经为你导入）。运行以下单元格来帮助你调试：

8.1 损失函数合理性检查

当你构建一个新的网络后，首先应该做的事情之一就是对损失函数进行合理性检查。当我们使用 softmax 损失时，对于随机权重（且没有正则化），我们期望损失值大约为 $\log(C)$ ，其中 C 是类别数量。当我们添加正则化后，损失值应该会略有上升。

```
[18]: # 实例化一个三层卷积网络模型
model = ThreeLayerConvNet()

# 生成随机输入数据和标签
N = 50 # 样本数量
X = np.random.randn(N, 3, 32, 32) # 输入数据，形状为（样本数，通道数，高度，宽度）
y = np.random.randint(10, size=N) # 随机标签，取值范围为 0-9（共 10 个类别）

# 在没有正则化的情况下计算损失和梯度
loss, grads = model.loss(X, y)
print('初始损失 (无正则化): ', loss)

# 设置正则化强度
```

```

model.reg = 0.5
# 在有正则化的情况下计算损失和梯度
loss, grads = model.loss(X, y)
print('初始损失 (有正则化): ', loss)

```

初始损失 (无正则化): 2.302586071243987

初始损失 (有正则化): 2.508255638232932

8.2 梯度检查

在损失看起来合理之后，使用数值梯度检查来确保你的反向传播是正确的。进行数值梯度检查时，你应该使用少量的人工数据以及每一层使用少量的神经元。注意：正确的实现可能仍会有高达 $1e-2$ 量级的相对误差。

```

[19]: num_inputs = 2 # 输入样本数量
input_dim = (3, 16, 16) # 输入数据维度: (通道数, 高度, 宽度)
reg = 0.0 # 正则化强度
num_classes = 10 # 类别数量
np.random.seed(231) # 设置随机种子, 确保结果可复现
X = np.random.randn(num_inputs, *input_dim) # 生成随机输入数据
y = np.random.randint(num_classes, size=num_inputs) # 生成随机标签

# 实例化三层卷积网络模型
model = ThreeLayerConvNet(
    num_filters=3, # 卷积层的滤波器数量
    filter_size=3, # 滤波器大小
    input_dim=input_dim, # 输入数据维度
    hidden_dim=7, # 全连接隐藏层的维度
    dtype=np.float64 # 数据类型
)
# 计算损失和梯度
loss, grads = model.loss(X, y)

# 误差应该较小, 但正确的实现可能会有高达  $1e-2$  量级的相对误差
for param_name in sorted(grads):
    # 定义一个仅返回损失的函数 (用于数值梯度计算)
    f = lambda _: model.loss(X, y)[0]
    # 计算参数的数值梯度

```

```

    param_grad_num = eval_numerical_gradient(f, model.params[param_name],
↪ verbose=False, h=1e-6)
    # 计算相对误差
    e = rel_error(param_grad_num, grads[param_name])
    print('%s的最大相对误差: %e' % (param_name, rel_error(param_grad_num,
↪ grads[param_name])))

```

W1 的最大相对误差: 3.053965e-04

W2 的最大相对误差: 1.822723e-02

W3 的最大相对误差: 3.422399e-04

b1 的最大相对误差: 3.397321e-06

b2 的最大相对误差: 2.517459e-03

b3 的最大相对误差: 9.711800e-10

8.3 过拟合小数据集

一个不错的技巧是用少量训练样本训练你的模型。你应该能够过拟合小数据集，这会导致非常高的训练准确率和相对较低的验证准确率。

```

[20]: np.random.seed(231) # 设置随机种子, 保证结果可复现

num_train = 100 # 训练样本数量
# 构建小规模数据集
small_data = {
    'X_train': data['X_train'][:num_train], # 取前 100 个训练样本
    'y_train': data['y_train'][:num_train], # 对应的训练标签
    'X_val': data['X_val'], # 验证集数据
    'y_val': data['y_val'], # 验证集标签
}

# 实例化三层卷积网络模型, 权重缩放因子为 1e-2
model = ThreeLayerConvNet(weight_scale=1e-2)

# 实例化求解器 (用于模型训练)
solver = Solver(
    model, # 要训练的模型
    small_data, # 训练数据
    num_epochs=15, # 训练轮数

```

```

    batch_size=50, # 批处理大小
    update_rule='adam', # 优化更新规则 (使用 Adam)
    optim_config={'learning_rate': 1e-3}, # 优化器配置 (学习率为 1e-3)
    verbose=True, # 训练过程中打印详细信息
    print_every=1 # 每迭代 1 次打印一次信息
)

solver.train() # 执行训练

```

```

(迭代 1 / 30) 损失: 2.414060
(Epoch 0 / 15) 训练准确率: 0.200000; 验证准确率: 0.137000
(迭代 2 / 30) 损失: 3.102719
(Epoch 1 / 15) 训练准确率: 0.140000; 验证准确率: 0.087000
(迭代 3 / 30) 损失: 2.270332
(迭代 4 / 30) 损失: 2.099074
(Epoch 2 / 15) 训练准确率: 0.230000; 验证准确率: 0.093000
(迭代 5 / 30) 损失: 1.841253
(迭代 6 / 30) 损失: 1.935296
(Epoch 3 / 15) 训练准确率: 0.490000; 验证准确率: 0.168000
(迭代 7 / 30) 损失: 1.828834
(迭代 8 / 30) 损失: 1.652295
(Epoch 4 / 15) 训练准确率: 0.520000; 验证准确率: 0.182000
(迭代 9 / 30) 损失: 1.332530
(迭代 10 / 30) 损失: 1.772358
(Epoch 5 / 15) 训练准确率: 0.640000; 验证准确率: 0.171000
(迭代 11 / 30) 损失: 1.029629
(迭代 12 / 30) 损失: 1.038692
(Epoch 6 / 15) 训练准确率: 0.720000; 验证准确率: 0.226000
(迭代 13 / 30) 损失: 1.152896
(迭代 14 / 30) 损失: 0.834351
(Epoch 7 / 15) 训练准确率: 0.810000; 验证准确率: 0.248000
(迭代 15 / 30) 损失: 0.584665
(迭代 16 / 30) 损失: 0.644552
(Epoch 8 / 15) 训练准确率: 0.830000; 验证准确率: 0.240000
(迭代 17 / 30) 损失: 0.811508
(迭代 18 / 30) 损失: 0.430228
(Epoch 9 / 15) 训练准确率: 0.840000; 验证准确率: 0.175000

```

(迭代 19 / 30) 损失: 0.421580
(迭代 20 / 30) 损失: 0.555581
(Epoch 10 / 15) 训练准确率: 0.930000; 验证准确率: 0.197000
(迭代 21 / 30) 损失: 0.364954
(迭代 22 / 30) 损失: 0.271303
(Epoch 11 / 15) 训练准确率: 0.860000; 验证准确率: 0.215000
(迭代 23 / 30) 损失: 0.405969
(迭代 24 / 30) 损失: 0.385744
(Epoch 12 / 15) 训练准确率: 0.950000; 验证准确率: 0.207000
(迭代 25 / 30) 损失: 0.109979
(迭代 26 / 30) 损失: 0.113894
(Epoch 13 / 15) 训练准确率: 0.960000; 验证准确率: 0.218000
(迭代 27 / 30) 损失: 0.123754
(迭代 28 / 30) 损失: 0.172242
(Epoch 14 / 15) 训练准确率: 0.990000; 验证准确率: 0.222000
(迭代 29 / 30) 损失: 0.123923
(迭代 30 / 30) 损失: 0.070897
(Epoch 15 / 15) 训练准确率: 0.990000; 验证准确率: 0.218000

```
[21]: # 打印最终的训练准确率
print(
    "小数据集训练准确率: ",
    solver.check_accuracy(small_data['X_train'], small_data['y_train']) # 检查
    模型在小数据集训练集上的准确率
)
```

小数据集训练准确率: 0.81

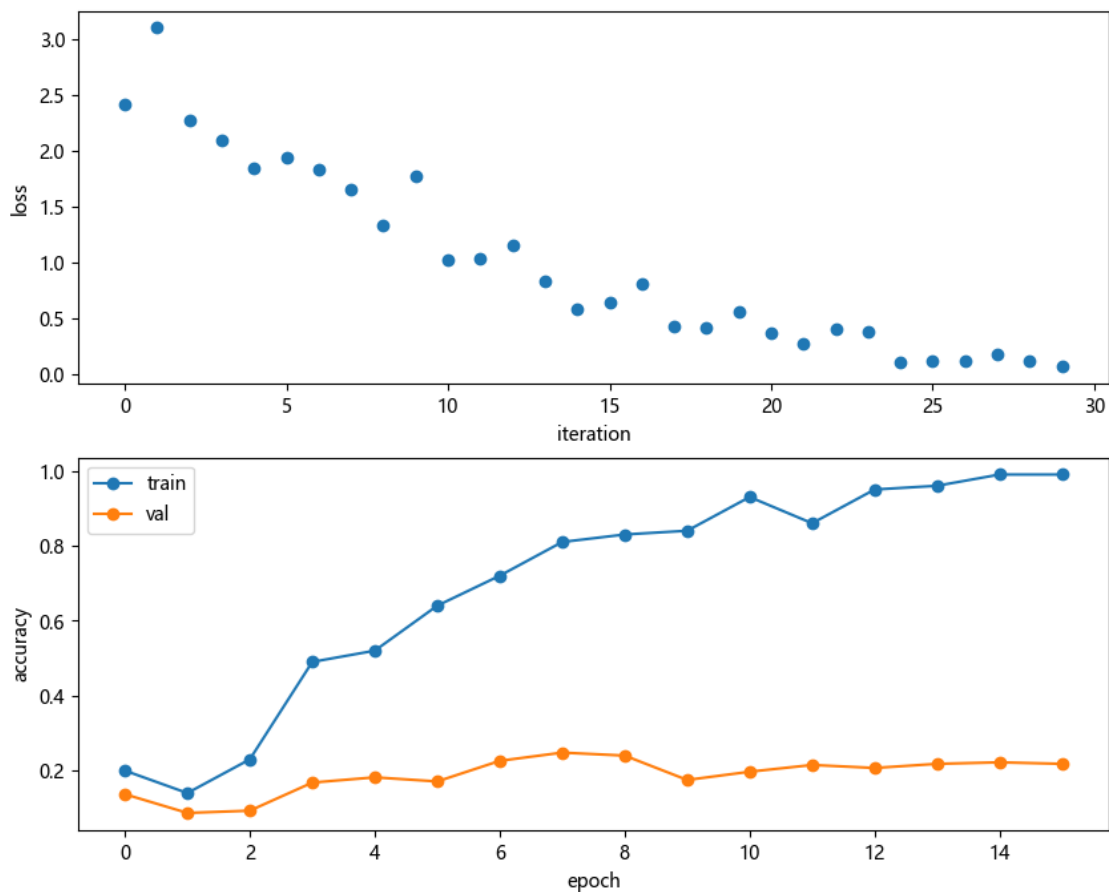
```
[22]: # 打印最终的验证准确率
print(
    "小数据集验证准确率: ",
    solver.check_accuracy(small_data['X_val'], small_data['y_val']) # 检查模型
    在小数据集验证集上的准确率
)
```

小数据集验证准确率: 0.248

绘制损失、训练准确率和验证准确率的图表，应该能明显看出过拟合现象：

```
[23]: # 创建一个 2 行 1 列的子图, 当前绘制第 1 个子图
plt.subplot(2, 1, 1)
# 绘制损失历史, 用圆点标记每个点
plt.plot(solver.loss_history, 'o')
# 设置 x 轴标签为 "迭代次数"
plt.xlabel('iteration')
# 设置 y 轴标签为 "损失"
plt.ylabel('loss')

# 当前绘制第 2 个子图
plt.subplot(2, 1, 2)
# 绘制训练准确率历史, 用线和圆点标记
plt.plot(solver.train_acc_history, '-o')
# 绘制验证准确率历史, 用线和圆点标记
plt.plot(solver.val_acc_history, '-o')
# 添加图例, "train" 和 "val" 分别对应训练和验证准确率, 位置在左上角
plt.legend(['train', 'val'], loc='upper left')
# 设置 x 轴标签为 "轮次"
plt.xlabel('epoch')
# 设置 y 轴标签为 "准确率"
plt.ylabel('accuracy')
# 显示图形
plt.show()
```



8.4 训练网络

通过用一个轮次训练这个三层卷积网络，你应该能在训练集上达到 40% 以上的准确率：

```
[24]: # 实例化三层卷积网络模型
# weight_scale=0.001: 权重初始化的缩放因子
# hidden_dim=500: 全连接隐藏层的维度为 500
# reg=0.001: 正则化强度为 0.001
model = ThreeLayerConvNet(weight_scale=0.001, hidden_dim=500, reg=0.001)

# 创建求解器用于模型训练
solver = Solver(
    model, # 要训练的模型
    data, # 训练数据
```

```

num_epochs=1, # 训练轮数为 1
batch_size=50, # 批处理大小为 50
update_rule='adam', # 使用 Adam 优化算法
optim_config={'learning_rate': 1e-3}, # 优化器配置, 学习率为 0.001
verbose=True, # 训练过程中打印详细信息
print_every=20 # 每 20 次迭代打印一次信息
)
solver.train() # 执行训练

```

```

(迭代 1 / 980) 损失: 2.304740
(Epoch 0 / 1) 训练准确率: 0.103000; 验证准确率: 0.107000
(迭代 21 / 980) 损失: 2.129645
(迭代 41 / 980) 损失: 1.945061
(迭代 61 / 980) 损失: 1.770560
(迭代 81 / 980) 损失: 1.901148
(迭代 101 / 980) 损失: 1.919195
(迭代 121 / 980) 损失: 1.765398
(迭代 141 / 980) 损失: 1.898770
(迭代 161 / 980) 损失: 1.738502
(迭代 181 / 980) 损失: 1.843249
(迭代 201 / 980) 损失: 2.066862
(迭代 221 / 980) 损失: 1.990459
(迭代 241 / 980) 损失: 1.828815
(迭代 261 / 980) 损失: 1.604638
(迭代 281 / 980) 损失: 1.692068
(迭代 301 / 980) 损失: 1.754931
(迭代 321 / 980) 损失: 1.742302
(迭代 341 / 980) 损失: 1.699758
(迭代 361 / 980) 损失: 1.723792
(迭代 381 / 980) 损失: 1.542635
(迭代 401 / 980) 损失: 1.738328
(迭代 421 / 980) 损失: 1.427555
(迭代 441 / 980) 损失: 1.759026
(迭代 461 / 980) 损失: 1.879609
(迭代 481 / 980) 损失: 1.413088
(迭代 501 / 980) 损失: 1.367136
(迭代 521 / 980) 损失: 1.637779

```


(迭代 541 / 980) 损失: 1.749722
(迭代 561 / 980) 损失: 1.645201
(迭代 581 / 980) 损失: 1.373341
(迭代 601 / 980) 损失: 1.573957
(迭代 621 / 980) 损失: 1.570977
(迭代 641 / 980) 损失: 1.683598
(迭代 661 / 980) 损失: 1.755739
(迭代 681 / 980) 损失: 1.741579
(迭代 701 / 980) 损失: 1.627704
(迭代 721 / 980) 损失: 1.655453
(迭代 741 / 980) 损失: 1.634793
(迭代 761 / 980) 损失: 1.447560
(迭代 781 / 980) 损失: 1.793672
(迭代 801 / 980) 损失: 1.627485
(迭代 821 / 980) 损失: 1.656835
(迭代 841 / 980) 损失: 1.351238
(迭代 861 / 980) 损失: 1.745392
(迭代 881 / 980) 损失: 1.552356
(迭代 901 / 980) 损失: 1.539406
(迭代 921 / 980) 损失: 1.492501
(迭代 941 / 980) 损失: 1.683674
(迭代 961 / 980) 损失: 1.638435
(Epoch 1 / 1) 训练准确率: 0.480000; 验证准确率: 0.485000

```
[25]: # 打印最终的训练准确率
print(
    "完整数据集训练准确率: ",
    solver.check_accuracy(data['X_train'], data['y_train']) # 检查模型在完整训练
    集上的准确率
)
```

完整数据集训练准确率: 0.46477551020408164

```
[26]: # 打印最终的验证准确率
print(
    "完整数据集验证准确率: ",
    solver.check_accuracy(data['X_val'], data['y_val']) # 检查模型在完整验证集上
    的准确率
)
```

```
)
```

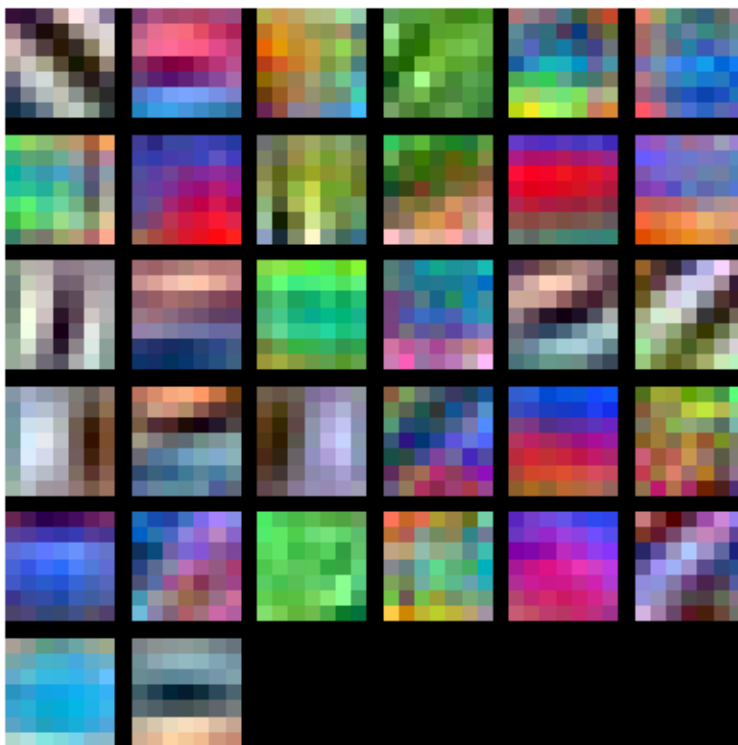
完整数据集验证准确率： 0.485

8.5 可视化滤波器

你可以通过运行以下代码来可视化训练好的网络中第一层的卷积滤波器：

```
[27]: from cs231n.vis_utils import visualize_grid # 从可视化工具中导入可视化网格函数

# 可视化第一层卷积权重：将权重的维度从（滤波器数，输入通道，高，宽）转置为（滤波器数，高，宽，输入通道），然后生成网格图像
grid = visualize_grid(model.params['W1'].transpose(0, 2, 3, 1))
plt.imshow(grid.astype('uint8')) # 显示图像，转换为无符号 8 位整数格式
plt.axis('off') # 关闭坐标轴显示
plt.gcf().set_size_inches(5, 5) # 设置当前图形的大小为 5x5 英寸
plt.show() # 显示图形
```



9 空间批量归一化

我们已经了解到，批量归一化是训练深度全连接网络时非常有用的技术。正如原始论文（链接见 `BatchNormalization.ipynb`）中所提出的，批量归一化也可用于卷积网络，但需要稍作调整；这种修改后的方法被称为“空间批量归一化”。

通常，批量归一化接收形状为 (N, D) 的输入，并生成形状为 (N, D) 的输出，其中我们会沿着小批量维度 N 进行归一化。对于来自卷积层的数据，批量归一化需要接收形状为 (N, C, H, W) 的输入，并生成形状为 (N, C, H, W) 的输出，其中 N 维度表示小批量大小， (H, W) 维度表示特征图的空间大小。

如果特征图是通过卷积生成的，那么我们期望每个特征通道的统计量（例如均值、方差）在不同图像之间以及同一图像的不同位置之间都相对一致——毕竟，每个特征通道都是由相同的卷积滤波器生成的！因此，空间批量归一化通过对小批量维度 N 以及空间维度 H 和 W 计算统计量，来为 C 个特征通道中的每一个通道计算均值和方差。

[1] Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”, ICML 2015.

10 空间批量归一化：前向传播

在文件 `cs231n/layers.py` 中，在 `spatial_batchnorm_forward` 函数里实现空间批量归一化的前向传播。运行以下代码来检查你的实现：

```
[28]: np.random.seed(231) # 设置随机种子，确保结果可复现

# 通过检查空间批量归一化前后特征的均值和方差，验证训练时的前向传播
N, C, H, W = 2, 3, 4, 5 # N: 批量大小, C: 通道数, H: 高度, W: 宽度
x = 4 * np.random.randn(N, C, H, W) + 10 # 生成随机输入数据，均值约为 10，标准差约为 4

print('空间批量归一化之前：')
print('  形状：', x.shape)
print('  均值：', x.mean(axis=(0, 2, 3))) # 计算每个通道在 (N, H, W) 维度上的均值
print('  标准差：', x.std(axis=(0, 2, 3))) # 计算每个通道在 (N, H, W) 维度上的标准差

# 归一化后，均值应接近 0，标准差应接近 1
gamma, beta = np.ones(C), np.zeros(C) # gamma 设为全 1, beta 设为全 0
```

```

bn_param = {'mode': 'train'} # 批量归一化参数, 设置为训练模式
out, _ = spatial_batchnorm_forward(x, gamma, beta, bn_param) # 执行空间批量归一化前向传播
print('空间批量归一化之后: ')
print(' 形状: ', out.shape)
print(' 均值: ', out.mean(axis=(0, 2, 3)))
print(' 标准差: ', out.std(axis=(0, 2, 3)))

# 使用非平凡的 gamma 和 beta 时, 均值应接近 beta, 标准差应接近 gamma
gamma, beta = np.asarray([3, 4, 5]), np.asarray([6, 7, 8]) # 自定义 gamma 和 beta
out, _ = spatial_batchnorm_forward(x, gamma, beta, bn_param)
print('空间批量归一化之后 (使用非平凡的 gamma、beta): ')
print(' 形状: ', out.shape)
print(' 均值: ', out.mean(axis=(0, 2, 3)))
print(' 标准差: ', out.std(axis=(0, 2, 3)))

```

空间批量归一化之前:

```

形状: (2, 3, 4, 5)
均值: [9.33463814 8.90909116 9.11056338]
标准差: [3.61447857 3.19347686 3.5168142 ]

```

空间批量归一化之后:

```

形状: (2, 3, 4, 5)
均值: [ 6.18949336e-16  5.99520433e-16 -1.22124533e-16]
标准差: [0.99999962 0.99999951 0.9999996 ]

```

空间批量归一化之后 (使用非平凡的 gamma、beta):

```

形状: (2, 3, 4, 5)
均值: [6. 7. 8.]
标准差: [2.99999885 3.99999804 4.99999798]

```

[29]: `np.random.seed(231)` # 设置随机种子, 保证结果可复现

```

# 验证测试时的前向传播: 先运行多次训练时的前向传播来预热滑动平均,
# 然后检查测试时前向传播后激活值的均值和方差
N, C, H, W = 10, 4, 11, 12 # N: 批量大小, C: 通道数, H: 高度, W: 宽度

bn_param = {'mode': 'train'} # 批量归一化参数, 初始设为训练模式

```

```

gamma = np.ones(C) # 缩放参数设为全 1
beta = np.zeros(C) # 偏移参数设为全 0

# 多次运行训练模式的前向传播，更新运行时的均值和方差（预热）
for t in range(50):
    x = 2.3 * np.random.randn(N, C, H, W) + 13 # 生成随机输入数据
    spatial_batchnorm_forward(x, gamma, beta, bn_param)

bn_param['mode'] = 'test' # 将批量归一化切换到测试模式
x = 2.3 * np.random.randn(N, C, H, W) + 13 # 生成新的随机输入数据
a_norm, _ = spatial_batchnorm_forward(x, gamma, beta, bn_param) # 执行测试模式
                        的前向传播

# 测试时的均值应接近 0，标准差应接近 1，但会比训练时的前向传播结果更嘈杂一些
print('空间批量归一化之后（测试时）：')
print('  均值：', a_norm.mean(axis=(0, 2, 3)))
print('  标准差：', a_norm.std(axis=(0, 2, 3)))

```

空间批量归一化之后（测试时）：

均值： [-0.08034406 0.07562881 0.05716371 0.04378383]

标准差： [0.96718744 1.0299714 1.02887624 1.00585577]

11 空间批量归一化：反向传播

在文件 `cs231n/layers.py` 中，在 `spatial_batchnorm_backward` 函数里实现空间批量归一化的反向传播。运行以下代码，使用数值梯度检查来验证你的实现：

```

[30]: np.random.seed(231) # 设置随机种子，确保结果可复现
N, C, H, W = 2, 3, 4, 5 # N: 批量大小, C: 通道数, H: 高度, W: 宽度
x = 5 * np.random.randn(N, C, H, W) + 12 # 生成随机输入数据
gamma = np.random.randn(C) # 随机初始化缩放参数 gamma
beta = np.random.randn(C) # 随机初始化偏移参数 beta
dout = np.random.randn(N, C, H, W) # 生成随机的输出梯度

bn_param = {'mode': 'train'} # 批量归一化参数，设为训练模式

# 定义用于数值梯度计算的函数

```

```

# 输入  $x$  的函数: 返回空间批量归一化的输出
fx = lambda x: spatial_batchnorm_forward(x, gamma, beta, bn_param)[0]
# 输入  $\gamma$  的函数: 返回空间批量归一化的输出 (注意这里变量名  $a$  实际指代  $\gamma$ )
fg = lambda a: spatial_batchnorm_forward(x, gamma, beta, bn_param)[0]
# 输入  $\beta$  的函数: 返回空间批量归一化的输出 (注意这里变量名  $b$  实际指代  $\beta$ )
fb = lambda b: spatial_batchnorm_forward(x, gamma, beta, bn_param)[0]

# 计算  $x$ 、 $\gamma$ 、 $\beta$  的数值梯度
dx_num = eval_numerical_gradient_array(fx, x, dout)
da_num = eval_numerical_gradient_array(fg, gamma, dout)
db_num = eval_numerical_gradient_array(fb, beta, dout)

# 你应该期望误差量级在  $1e-12$  到  $1e-06$  之间
# 执行前向传播获取缓存
_, cache = spatial_batchnorm_forward(x, gamma, beta, bn_param)
# 执行反向传播计算解析梯度
dx, dgamma, dbeta = spatial_batchnorm_backward(dout, cache)

# 打印数值梯度与解析梯度之间的相对误差
print('dx 误差: ', rel_error(dx_num, dx))
print('dgamma 误差: ', rel_error(da_num, dgamma))
print('dbeta 误差: ', rel_error(db_num, dbeta))

```

dx 误差: 2.786648197756335e-07

dgamma 误差: 7.0974817113608705e-12

dbeta 误差: 3.275608725278405e-12

12 空间组归一化

在上一个笔记本中, 我们提到层归一化是另一种归一化技术, 它减轻了批量归一化在批量大小方面的限制。然而, 正如文献 [2] 的作者所观察到的, 当层归一化用于卷积层时, 其性能不如批量归一化:

对于全连接层来说, 一层中的所有隐藏单元对最终预测的贡献往往相似, 因此对一层的输入总和进行重新中心化和重新缩放效果很好。但是, 对于卷积神经网络, 这种贡献相似的假设不再成立。大量感受野位于图像边界附近的隐藏单元很少被激活, 因此与同一层中其他隐藏单元的统计特性有很大差异。

文献 [3] 的作者提出了一种中间技术。与层归一化 (对每个数据点的整个特征进行归一化) 不同, 他们建议将每个数据点的特征一致地分成 G 个组, 然后对每个组、每个数据点进行归一化。

迄今为止讨论的归一化技术的直观对比 (图像经文献 [3] 编辑而成)

尽管仍然假设每个组内的贡献是均等的, 但作者推测这一问题并不严重, 因为在视觉识别中, 特征内部会自然形成分组。他们用来举例说明这一点的一个例子是, 在传统计算机视觉中, 许多高性能的手工特征都有明确分组的项。例如方向梯度直方图 (HOG) [4]——在计算每个空间局部块的直方图后, 每个块的直方图在被连接起来形成最终特征向量之前都会被归一化。

现在你要实现组归一化。

[2] Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. “Layer Normalization.” *stat* 1050 (2016): 21.

[3] Wu, Yuxin, and Kaiming He. “Group Normalization.” *arXiv preprint arXiv:1803.08494* (2018).

[4] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *Computer Vision and Pattern Recognition (CVPR)*, 2005.

13 空间组归一化：前向传播

在文件 `cs231n/layers.py` 中, 在 `spatial_groupnorm_forward` 函数里实现组归一化的前向传播。运行以下代码来检查你的实现:

```
[31]: np.random.seed(231) # 设置随机种子, 确保结果可复现

# 通过检查空间组归一化前后特征的均值和方差, 验证训练时的前向传播
N, C, H, W = 2, 6, 4, 5 # N: 批量大小, C: 通道数, H: 高度, W: 宽度
G = 2 # 组数, 将特征分成 2 个组
x = 4 * np.random.randn(N, C, H, W) + 10 # 生成随机输入数据, 均值约为 10, 标准差约为 4
x_g = x.reshape((N*G, -1)) # 将输入重塑为 (每组样本数, 每组特征数) 的形状, 便于计算每组的统计量
print('空间组归一化之前: ')
print(' 形状: ', x.shape)
print(' 均值: ', x_g.mean(axis=1)) # 计算每组的均值 (沿特征维度)
print(' 标准差: ', x_g.std(axis=1)) # 计算每组的标准差 (沿特征维度)

# 归一化后, 均值应接近 0, 标准差应接近 1
```

```

gamma, beta = np.ones((1, C, 1, 1)), np.zeros((1, C, 1, 1)) # gamma 设为全 1,
beta 设为全 0, 维度为 (1, C, 1, 1) 以适配广播
bn_param = {'mode': 'train'} # 组归一化参数, 设置为训练模式

out, _ = spatial_groupnorm_forward(x, gamma, beta, G, bn_param) # 执行空间组归
一化前向传播
out_g = out.reshape((N*G, -1)) # 将输出重塑为 (每组样本数, 每组特征数) 的形状
print('空间组归一化之后: ')
print(' 形状: ', out.shape)
print(' 均值: ', out_g.mean(axis=1)) # 计算归一化后每组的均值
print(' 标准差: ', out_g.std(axis=1)) # 计算归一化后每组的标准差

```

空间组归一化之前:

形状: (2, 6, 4, 5)

均值: [9.72505327 8.51114185 8.9147544 9.43448077]

标准差: [3.67070958 3.09892597 4.27043622 3.97521327]

空间组归一化之后:

形状: (2, 6, 4, 5)

均值: [-2.14643118e-16 5.25505565e-16 2.65528340e-16 -3.38618023e-16]

标准差: [0.99999963 0.99999948 0.99999973 0.99999968]

14 空间组归一化: 反向传播

在文件 `cs231n/layers.py` 中, 在 `spatial_groupnorm_backward` 函数里实现空间组归一化的反向传播。运行以下代码, 使用数值梯度检查来验证你的实现:

```

[32]: np.random.seed(231) # 设置随机种子, 确保结果可复现
N, C, H, W = 2, 6, 4, 5 # N: 批量大小, C: 通道数, H: 高度, W: 宽度
G = 2 # 组数, 将特征分为 2 个组
x = 5 * np.random.randn(N, C, H, W) + 12 # 生成随机输入数据
gamma = np.random.randn(1, C, 1, 1) # 随机初始化缩放参数 gamma, 维度为 (1, C, 1, 1) 以适配广播
beta = np.random.randn(1, C, 1, 1) # 随机初始化偏移参数 beta, 维度为 (1, C, 1, 1) 以适配广播
dout = np.random.randn(N, C, H, W) # 生成随机的输出梯度

gn_param = {} # 组归一化参数 (空字典, 此处无需额外配置)

```



```

# 定义用于数值梯度计算的函数
# 输入  $x$  的函数: 返回空间组归一化的输出
fx = lambda x: spatial_groupnorm_forward(x, gamma, beta, G, gn_param)[0]
# 输入  $gamma$  的函数: 返回空间组归一化的输出 (注意这里变量名  $a$  实际指代  $gamma$ )
fg = lambda a: spatial_groupnorm_forward(x, gamma, beta, G, gn_param)[0]
# 输入  $beta$  的函数: 返回空间组归一化的输出 (注意这里变量名  $b$  实际指代  $beta$ )
fb = lambda b: spatial_groupnorm_forward(x, gamma, beta, G, gn_param)[0]

# 计算  $x$ 、 $gamma$ 、 $beta$  的数值梯度
dx_num = eval_numerical_gradient_array(fx, x, dout)
da_num = eval_numerical_gradient_array(fg, gamma, dout)
db_num = eval_numerical_gradient_array(fb, beta, dout)

# 执行前向传播获取缓存
_, cache = spatial_groupnorm_forward(x, gamma, beta, G, gn_param)
# 执行反向传播计算解析梯度
dx, dgamma, dbeta = spatial_groupnorm_backward(dout, cache)

# 你应该期望误差量级在  $1e-12$  到  $1e-07$  之间
print('dx 误差: ', rel_error(dx_num, dx))
print('dgamma 误差: ', rel_error(da_num, dgamma))
print('dbeta 误差: ', rel_error(db_num, dbeta))

```

```

dx 误差: 7.413109404620405e-08
dgamma 误差: 9.468195772749234e-12
dbeta 误差: 3.354494437653335e-12

```